

Amazon Now – Pincode Serviceability Checker

Project Overview

This project automates the process of checking whether **Amazon Now** (Amazon's quick-commerce grocery delivery service) is serviceable for a given list of pincodes across India. The input is a dataset of **4,098 dark store locations** (across Blinkit, Swiggy Instamart, and Zepto), covering **1,549 unique pincodes**. The tool checks each pincode against Amazon Now and records whether the service is available or not.

Results Summary

Metric	Value
Total Rows in Input	4,098
Unique Pincodes Checked	1,549
Amazon Now Serviceable (YES)	400
Amazon Now NOT Serviceable (NO)	1,149
Errors	0
Accuracy (Manual Verification)	~96.7%

Approach & Methodology

Phase 1: Reverse Engineering the API

The initial approach was to **reverse-engineer Amazon's internal APIs** by inspecting network traffic:

- Inspected Network Requests:** Analyzed XHR/Fetch requests while changing pincodes on the Amazon Now storefront.
- Found Internal API Endpoints:** Extracted API keys, session tokens, and request headers.
- Built a Direct API Script:** Developed a Python script to query these APIs directly.

Challenge: Amazon's aggressive **anti-bot protection (AWS WAF)** flagged automated requests within minutes, resulting in `503 Service Unavailable` and `403 Forbidden` errors due to missing browser fingerprints (TLS, canvas, WebGL) and suspicious request rates.

Phase 2: Browser Automation with Playwright

To bypass the WAF restrictions, the approach shifted to **browser automation using Playwright**, simulating genuine human interaction in a real Chrome browser:

1. **Login Session Persistence:** The script opens a real browser for manual login on the first run, saving cookies to a local session file to maintain authenticated state without repeated logins.
 2. **Pincode Injection & Verification:** For each pincode, the script:
 3. Navigates to the Amazon homepage.
 4. Interacts with the "Deliver to" location selector.
 5. Types the pincode and applies it.
 6. **Crucially verifies** that the pincode has successfully updated in the UI to prevent race conditions.
 7. **Serviceability Detection:** Navigates to the Amazon Now storefront and scans the DOM for positive signals (product cards, "Add to Cart" buttons) and negative signals ("Not available in your area"). A weighted scoring system determines the final result.
-

Challenges & Solutions

1. Bot Detection & Rate Limiting

- **Problem:** Traditional scraping was immediately blocked.
- **Solution:** Deployed Playwright to pass browser fingerprinting. Implemented randomized human-like delays (1-3 seconds) and realistic typing speeds (40ms per keystroke).

2. UI Race Conditions

- **Problem:** The location modal would close, but the pincode wouldn't always update before the script navigated to the storefront, causing false positives/negatives.
- **Solution:** Added strict verification logic. The script explicitly reads the active location text from the navbar and confirms the new pincode is active before proceeding.

3. Dynamic Element Selectors

- **Problem:** Amazon frequently changes element IDs or hides the location popup behind overlays.
- **Solution:** Implemented robust fallback selectors (`#nav-global-location-popover-link` → `#glow-ingress-line1`) and a self-healing retry mechanism that returns to the

homepage on failure.

4. Interrupted Execution

- **Problem:** Checking 1,500+ pincodes takes hours. Network drops or errors could ruin a run.
 - **Solution:** Engineered an incremental checkpointing system (`progress.json`). State is saved every 25 pincodes, and the script can resume exactly where it left off using the `--resume` flag.
-

Project Structure

```
|— amazon_now_checker.py # Main automation script |— quick commerce dark
store.xlsx # Input dataset (4,098 dark store pincodes) |— amazon_now_results.xlsx #
Output results and summary metrics |— README.md # Project documentation (Note:
amazon_session.json and progress.json are generated dynamically during execution.)
```

How to Run

Prerequisites

```
bash pip install playwright openpyxl playwright install chromium
```

Step 1: Login (First Time Only)

```
bash python amazon_now_checker.py --login
```

 Log into Amazon manually in the opened browser window, then press Enter in the terminal to save the session.

Step 2: Run the Checker

```
bash python amazon_now_checker.py
```

Additional Options

```
bash python amazon_now_checker.py --resume # Resume from last checkpoint python
amazon_now_checker.py --limit 50 # Check only first 50 pincodes
```

Key Learnings

1. **API Defenses are Robust:** Modern anti-bot systems (like AWS WAF) effectively neutralize raw HTTP scraping.

2. **Browser Automation as a Fallback:** Tools like Playwright are essential for scraping highly-defended targets, as they generate genuine browser fingerprints.
 3. **State Management is Critical:** For long-running scripts, incremental checkpointing ensures resilience against network/system failures.
 4. **UI Verification:** Always verify UI state changes explicitly before proceeding; relying solely on timeouts leads to race conditions.
-



Author

Developed for competitive analysis of quick-commerce geographic coverage.



Quality Assurance & Manual Validation

- **Problem:** Automated UI scraping can occasionally misread dynamic content, especially on complex sites like Amazon.
- **Solution:** To ensure the highest data integrity, a random sample of 30 processed pincodes was selected for **manual verification** by a human operator against the live Amazon Now website.
- **Result:** Only 1 out of the 30 sampled pincodes produced an incorrect result, yielding a **96.7% accuracy rate** in the sampled batch.
- **Margin of Error:** Extrapolating to the full dataset, we estimate a conservative **~5% margin of error** for the final output, accounting for potential intermittent UI variations or temporary location blocks during the scraping process.